

Diabetes Prediction

Kaggle Project Review Notes

Dataset: **Diabetes Data Set** (Kaggle, vikasukani). This is the first complete project that runs through cleaning, EDA, modeling, and tuning. Earlier exercises were isolated techniques; this chapter connects them into a repeatable diagnostic workflow: first understand what the data looks like, find hidden traps, build a runnable model, inspect where it fails, and then repair the specific weakness.

Python 3 - scikit-learn

tutor style

visual explanations

FAQ included

complete project pipeline

00 Project Pipeline and Review Order

The value of this project is not one isolated technique, but the complete chain that connects all of them.



Earlier chapters taught separate tools; this project connects them into an iterative chain.

Understand data -> find traps -> build a model -> diagnose errors -> target the repair.

01 - START

Fake zeros

domain knowledge

02

replace + fillna

order cannot reverse

03

EDA suite

distribution + groups

04

Non-convergence

scale problem

05

Scaler leakage

fit vs transform

06

Compare coef_

only after scaling

07

Multicollinearity

why rankings differ

08

Imbalance trap

accuracy misleads

09

class_weight

trade error costs

Connection to earlier chapters: missing-value handling uses Pandas, EDA uses Matplotlib concepts through Seaborn, scaling answers KNN's sensitivity to numeric range, and the accuracy trap plus **class_weight** extend precision/recall. The project introduces no exotic syntax; it develops diagnostic ability - knowing where to look when a model produces a pile of numbers.

01 Use Medical Knowledge to Detect Missing Values Disguised as Zero

Core concept

```
(diabetes_data == 0).sum()
```

Glucose: 13

BloodPressure: 90

Insulin: 956

Blood pressure of zero means a patient is dead -> these zeros are missing values.

Pregnancies = 0 is valid -> leave it unchanged.

The decision comes from the field's business or physical meaning, not a statistical anomaly rule.

Count zeros in every column first

inspect

```
(diabetes_data == 0).sum()
# Glucose          13
# BloodPressure    90
# SkinThickness    573
# Insulin          956
# BMI              28
```

This is not statistical anomaly detection such as Z-scores. Statistics cannot know that zero is impossible. Only field meaning tells you that **Glucose**, **BloodPressure**, **SkinThickness**, **Insulin**, and **BMI** cannot meaningfully be zero, while zero pregnancies is entirely plausible.

Q. Why cannot a statistical method such as a Z-score determine which zeros are missing?

A Z-score finds values far from other values, but a value can have a broad natural range, as Insulin does, without being invalid. The truly impossible zero is defined by the field's physical or business meaning. That is a domain judgment, not a statistical one.

Use case: on any real dataset, first check whether every column's observed range makes sense for that variable before running anomaly detection.

02 replace(0, NaN) Must Come Before fillna(median)

• Core concept

The order cannot be reversed

order matters

```
diabetes_data[cols] = diabetes_data[cols].replace(0, np.nan)
diabetes_data.fillna(diabetes_data.median(), inplace=True)
```

Convert invalid zeros to **NaN** first. Only then will **median()** skip those missing values and compute the median from genuine observations. If you compute the median before replacement, the fake zeros participate as if they were normal data and pull the median downward.

Use case: when a dataset encodes invalid observations with a known sentinel, first mark them as NaN and only then impute them. These steps cannot be reversed.

03 The Seaborn EDA Suite: Distribution -> Group Comparison -> Correlation

• Core concept

Four-part EDA suite

seaborn

```
sns.histplot(data=df, x=feature, kde=True)           # one-variable distribution
sns.boxplot(data=df, x="Outcome", y=feature, ax=ax)   # compare outcome groups
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")   # correlation matrix
sns.pairplot(df, hue="Outcome")                       # two-variable plots + class color
```

The general sequence is: first inspect the shape of each variable, then ask whether its distribution differs across outcome groups, and finally inspect relationships among all variables. These are direct applications of earlier line, boxplot, and heatmap ideas, now expressed with Seaborn's higher-level API.

Use case: for any labeled classification dataset, run this first-pass EDA before feature selection or modeling.

Why Logistic Regression May Not Converge

Core concept

$\text{new weight} = \text{old weight} - \text{learning rate} \times \text{error} \times \text{raw feature value}$

Insulin: 15 to 846

DiabetesPedigreeFunction: 0.08 to 2.4

A hundred-fold scale gap can create a thousand-fold gap in weight updates.

One coordinate oscillates while another crawls; 100 iterations may never reach the optimum.

Raw feature values enter directly into gradient-descent update magnitudes.

Gradient descent updates weights with an expression containing the raw feature value. Here, Insulin ranges from about 15 to 846 while DiabetesPedigreeFunction ranges from about 0.08 to 2.4. The same error and learning rate can therefore produce update magnitudes that differ by orders of magnitude, making the optimization path unstable.

Q. Why did ConvergenceWarning disappear immediately after scaling?

Non-convergence and scale mismatch are the same underlying problem, not two independent bugs. Inconsistent ranges make optimization oscillate; `StandardScaler` repairs the cause, so the warning disappears as a consequence. Increasing `max_iter` alone only gives the unstable path more time.

Use case: for any gradient-based model with `ConvergenceWarning`, first unify feature scales instead of reflexively increasing `max_iter`.

StandardScaler: fit_transform Training Data, transform Test Data

Core concept

Fit statistics on training data; reuse them on test data

scaler

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # learn mean/std and transform
X_test_scaled = scaler.transform(X_test)      # reuse training mean/std only
```

fit means calculate and store the column means and standard deviations. **transform** means apply those stored values. The test set must not be fit again; otherwise its distribution leaks into the preprocessing pipeline and makes evaluation look better than a real deployment can achieve.

Q. Why can the test set not fit its own scaler?

Fit statistics represent everything the model is allowed to know about the data distribution. A deployed model cannot inspect future data in advance. Fitting on the test set lets the pipeline peek at that future distribution, creating data leakage.

Use case: any standardization or normalization pipeline follows the same law: statistics come only from training data. PCA, normalization, and outlier clipping use the same principle.

model.coef_ Becomes Comparable Only After Scaling

Core concept

Before scaling, coefficient magnitude is contaminated by each feature's unit and numeric range. A large-range feature such as **Insulin** may receive a small coefficient, while a small-range feature such as **DiabetesPedigreeFunction** may receive a large one. After scaling, every feature is measured in one-standard-deviation units, so absolute coefficient sizes can reflect relative importance.

```
coef_series = pd.Series(model.coef_[0], index=X.columns)
coef_series.sort_values(key=abs, ascending=False)
# Glucose          1.03
# BMI              0.56
# Pregnancies      0.44
# DiabetesPedigreeFunction 0.27
# Age              0.15
# Insulin          -0.11
# SkinThickness    0.03
# BloodPressure    -0.01
```

Sort with key=abs. A negative sign indicates direction, not weakness. Sorting the raw signed values would incorrectly push large negative effects to the bottom.

Use case: coefficient-based importance analysis for a linear model requires scaling first.

07 Why Correlation Heatmaps and coef_ Rankings Differ

• Core concept

A heatmap asks about each variable's one-to-one relationship with **Outcome**. A model coefficient is computed while all variables enter together. If two features are correlated, such as **SkinThickness** and **BMI**, the model may let one stand in for the other and shrink the second coefficient even when its pairwise correlation is substantial. Here, Insulin's negative and small coefficient is likely explained by overlap with Glucose.

Q. If the heatmap and coef_ ranking disagree, is the calculation wrong?

No. Correlation measures one variable against the outcome in isolation. A coefficient measures the variable's remaining independent explanatory power after controlling for all other features. Different rankings are a normal consequence of multicollinearity.

Use case: when interpreting a linear model with several correlated predictors, reduced coefficient magnitude does not automatically mean the feature is useless.

08 Accuracy Can Mislead on Imbalanced Data

• Core concept

confusion_matrix(Y_test, X_test_prediction)

	predicted 0	predicted 1
true 0	238	25
true 1	58 missed	79

Outcome=1 recall is only 0.58: 58 true cases were missed.

Overall accuracy is 0.79, but the confusion matrix reveals the real problem.

```
confusion_matrix(Y_test, X_test_prediction)
classification_report(Y_test, X_test_prediction)
# [[238 25]
#  [ 58 79]]
# class  precision  recall  f1-score
# 0      0.80      0.90      0.85
# 1      0.76      0.58      0.66
```

Outcome=1 recall is only 0.58: among 137 truly diabetic patients, 58 were missed. In a medical setting, a missed case usually costs more than a false alarm, so accuracy hides the clinically important failure. The dataset is also roughly 2:1 imbalanced, so the majority class dominates the overall accuracy.

Use case: in any classification task where classes are not close to 1:1, always inspect class-specific precision and recall.

09 class_weight='balanced': Principle and Trade-Off

• Core concept

Penalize mistakes on the minority class more heavily

class_weight

```
model = LogisticRegression(class_weight="balanced")
```

During training, each mistake on the smaller class **Outcome=1** receives a larger loss weight. The model therefore pays more attention to the minority class instead of simply fitting the more numerous class.

Metric for Outcome=1	Unweighted	Balanced
Recall	0.58	0.77
Precision	0.76	0.63
Accuracy	0.79	0.77

Missed cases fall from 58 to 32, while false alarms rise from 25 to 62 and overall accuracy falls slightly. This is the precision-recall trade-off: accept more false alarms and follow-up testing to avoid overlooking truly diabetic patients.

Q. Why use class_weight='balanced' instead of directly oversampling the minority class?

class_weight leaves the data unchanged and only changes error weights in the loss. It is simple, needs no extra library, and does not discard information. Under- or oversampling can be useful, but may waste data, create duplicates, or leak information if performed before the split. For moderate imbalance, class weighting is a low-cost first step.

Use case: class imbalance plus high cost of missed minority cases - medical screening, risk control, or fault detection - makes class_weight='balanced' a practical first optimization.

10 Reusable Code Templates

Compress the first ten sections into one complete, reusable workflow.

Template 1 - medically impossible zeros

clean

```
cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[cols] = df[cols].replace(0, np.nan)
df.fillna(df.median(), inplace=True)
# The invalid-zero columns are dataset-specific and must be chosen by domain meaning.
```

Template 2 - four-part EDA suite

eda

```
sns.histplot(data=df, x=feature, kde=True)
sns.boxplot(data=df, x=label_col, y=feature)
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
sns.pairplot(df, hue=label_col)
```

```
X = df.drop(columns=label_col)
Y = df[label_col]
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, stratify=Y, random_state=2
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(class_weight="balanced")
model.fit(X_train_scaled, Y_train)
# stratify=Y keeps class proportions consistent across train and test.
```

```
pred = model.predict(X_test_scaled)
print(accuracy_score(Y_test, pred))
print(confusion_matrix(Y_test, pred))
print(classification_report(Y_test, pred))
```

11 Common Error Summary

Error	Why it fails	Correct approach
<code>plt.show</code> without parentheses	You only referenced the function object; notebooks may still render because the inline backend is automatic.	<code>plt.show()</code>
Edit code but do not rerun the cell	Notebook output is a snapshot of the last execution and does not update automatically.	Restart Kernel and Run All.
<code>accuracy_score(X_train_prediction, Y_train)</code>	The signature is <code>y_true</code> first, <code>y_pred</code> second. Accuracy happens to be symmetric, but precision and recall are not.	<code>accuracy_score(Y_train, X_train_prediction)</code>
<code>model.predict(X_test)</code> after scaling training data	The model learned weights for the standardized value range; raw test data produces unreliable results.	<code>model.predict(X_test_scaled)</code>

The most persistent project bug is failing to rerun changed code. First, `accuracy_score` was edited but the cell still showed the earlier `TypeError`. Later, `StandardScaler` was added but dependent cells were not rerun, so the displayed warning and accuracy still came from the unscaled version. Correct source code is not enough; execution order must match the code.

12 FAQ

Q. Why cannot a Z-score determine which zeros are missing?

A Z-score finds values far from other observations, but zero can be close to the distribution and still be physically impossible. The invalidity comes from field meaning, so this is domain judgment rather than statistical judgment.

Q. Why did `ConvergenceWarning` disappear immediately after scaling?

Scale mismatch caused the unstable optimization path. `StandardScaler` repaired the cause; the warning disappeared as a symptom. These are not two separate issues.

Q. Why can the test set not fit its own scaler?

Fitted mean and standard deviation represent knowledge about the data distribution. Letting the test set fit them leaks future information into evaluation.

Q. If the heatmap and coef_ranking disagree, is the calculation wrong?

No. The heatmap measures one variable at a time; model coefficients measure independent contribution while controlling for the other variables. Correlated features can therefore change the ranking.

Q. Why use class_weight='balanced' instead of immediately oversampling the minority class?

Class weighting adjusts the loss without changing the dataset. It is simple and avoids losing or duplicating records. Sampling methods can be useful, but require extra care and must be performed after the data split.

13 Exercises

• Practice problems

1. **M** Switch classifiers and compare coef_ with feature_importances_:

```
from sklearn.ensemble import RandomForestClassifier
```

Train a random forest on the same training data, print `feature_importances_`, and compare the ranking with the logistic-regression coefficients. Expected: both usually rank Glucose near the top, while other features may differ because a linear coefficient and tree importance define "importance" differently.

2. **M** Adjust the classification threshold instead of class_weight:

```
probs = model.predict_proba(X_test_scaled)[: , 1]
```

```
pred_custom = (probs >= 0.3).astype(int)
```

Change the default threshold from 0.5 to 0.3, rerun the classification report, and inspect Outcome=1 recall and precision. Expected: recall rises while precision falls. Explain that `class_weight` changes training, whereas threshold tuning changes the prediction rule; the two can be combined.

3. **M** Replace StandardScaler with MinMaxScaler:

```
from sklearn.preprocessing import MinMaxScaler
```

Scale features to [0, 1], retrain, and compare `ConvergenceWarning` and accuracy with the `StandardScaler` version. Expected: convergence also improves because scales are aligned; accuracy should be similar but not necessarily identical. Explain what each scaler changes mathematically.

14 Review Order and Chapter Checklist

• Must master

Use domain knowledge, not a Z-score, to decide whether zero means missing

`replace(0, NaN)` must precede `fillna(median)`

`fit_transform` only on training data; transform test data to prevent leakage

On imbalanced classes, inspect precision and recall rather than accuracy alone

• Recommended practice

Scale mismatch can cause gradient descent not to converge

Compare `model.coef_` only after scaling

Understand `class_weight='balanced'` and the precision-recall trade-off

• Next steps

Tree `feature_importances_` versus linear-model coefficients

Manual threshold tuning with `predict_proba`

Over- and under-sampling such as SMOTE, always after the split

• Chapter checklist

- ☐ I can identify which zero-valued columns represent missing data and explain why.
- ☐ I can explain why inconsistent feature scales disrupt logistic-regression convergence.
- ☐ I can explain why coefficients become comparable only after scaling.
- ☐ I can diagnose class imbalance with `confusion_matrix` and `classification_report`.
- ☐ I rerun dependent cells after editing code instead of trusting stale notebook output.
- ☐ I can write `replace zero -> NaN -> median imputation` in the correct order.
- ☐ I can distinguish `fit_transform` on training data from `transform` on test data.
- ☐ I can explain disagreement between pairwise correlation and multivariate coefficients.
- ☐ I can explain `class_weight='balanced'` and its precision-recall trade-off.

This is the first real end-to-end project. You now have a reusable diagnostic workflow: hidden data trap -> model cannot train correctly -> coefficients cannot be compared -> one accuracy number is untrustworthy -> targeted, low-cost repair. This workflow is more valuable than any single trick, and the next Kaggle dataset will probably contain the same kinds of problems.